

OUC26夏移动软件开发-实验1

姓名：马一诺 学号：24020007088

项目	内容
姓名和学号	马一诺, 24020007088
本实验属于哪门课程	中国海洋大学26夏《移动软件开发》
实验名称	实验1: 热身运动
GitHub源码链接	https://github.com/1-qaz-2-wsx/ouc26-mobile-dev/tree/main/lab1
博客链接	https://blog.csdn.net/2604_96828618/article/details/164025493?spm=1011.2415.3001.10575&sharefrom=mp_manage_link

一、实验内容

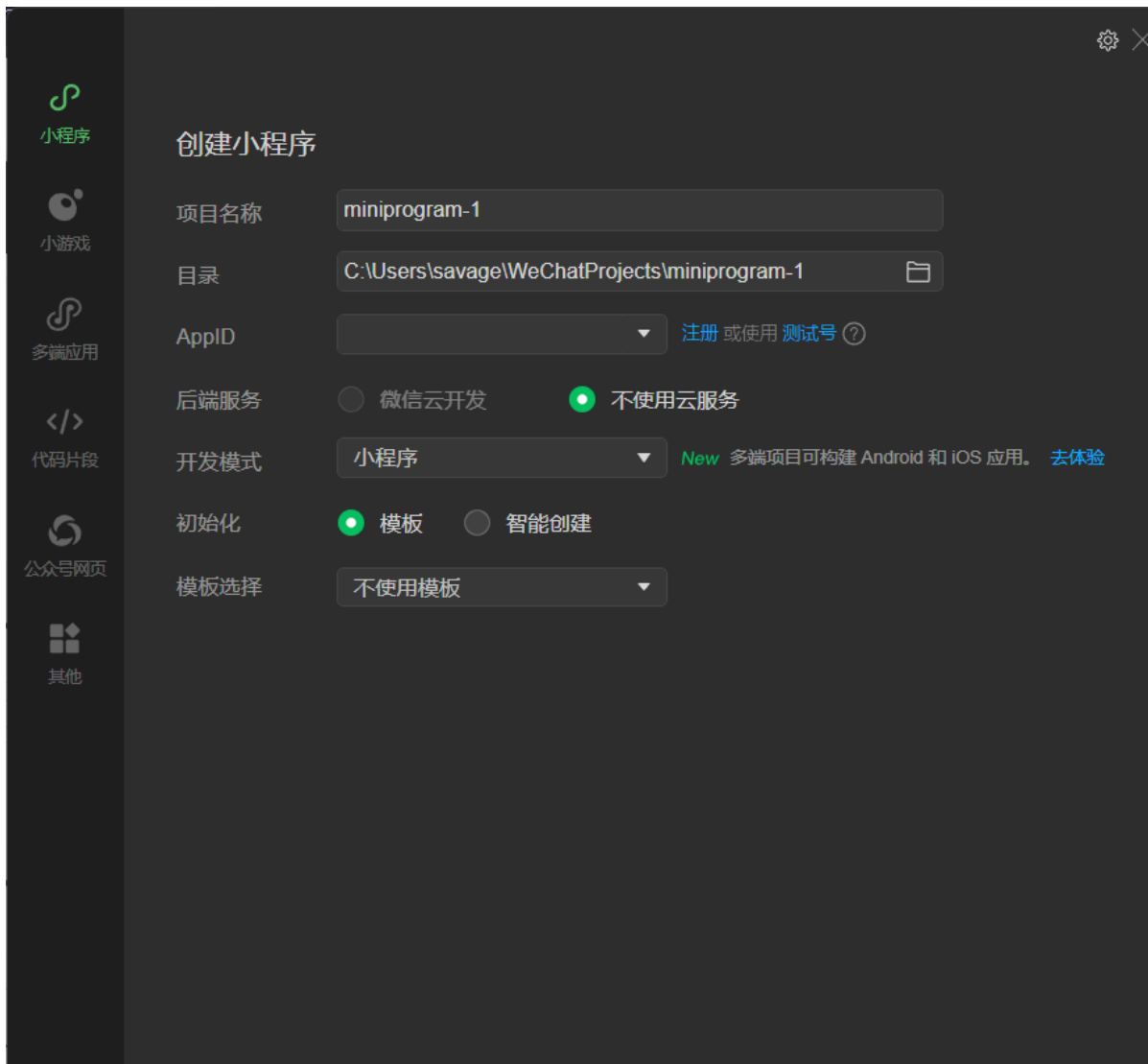
1. 安装微信开发者工具

首先访问[微信开发者工具官方下载页面](#)，根据计算机的操作系统下载稳定版安装包并完成安装。微信开发者工具提供了代码编辑、编译预览、模拟器和调试器等功能，是本实验开发和测试小程序的主要工具。



2. 创建第一个微信小程序项目

在本地创建空白项目目录，并在微信开发者工具中选择“小程序”项目类型，填写项目名称和目录后完成创建。本实验最终代码保存在 lab1/code1 文件夹中。



3. 设计“🐱 or 🐶”切换小程序

本实验在基础 Hello World 案例上进行扩展，制作一个可以循环切换小动物图片和问候文字的小程序。初始页面显示 `hello dog` 和腊肠犬图片，点击 `not me` 按钮后，按以下顺序切换：

1. `hello orange cat` 和橘猫图片；
2. `hello black cat` 和黑猫图片；
3. `hello white cat` 和白猫图片；
4. 再次回到 `hello dog` 和腊肠犬图片。

四张图片分别保存在 `code1/img` 文件夹中：

```
img/  
├─ sausage-hotdog.jpg  
├─ orange-cat.jpg  
├─ black-cat.jpg  
└─ white-cat.jpg
```

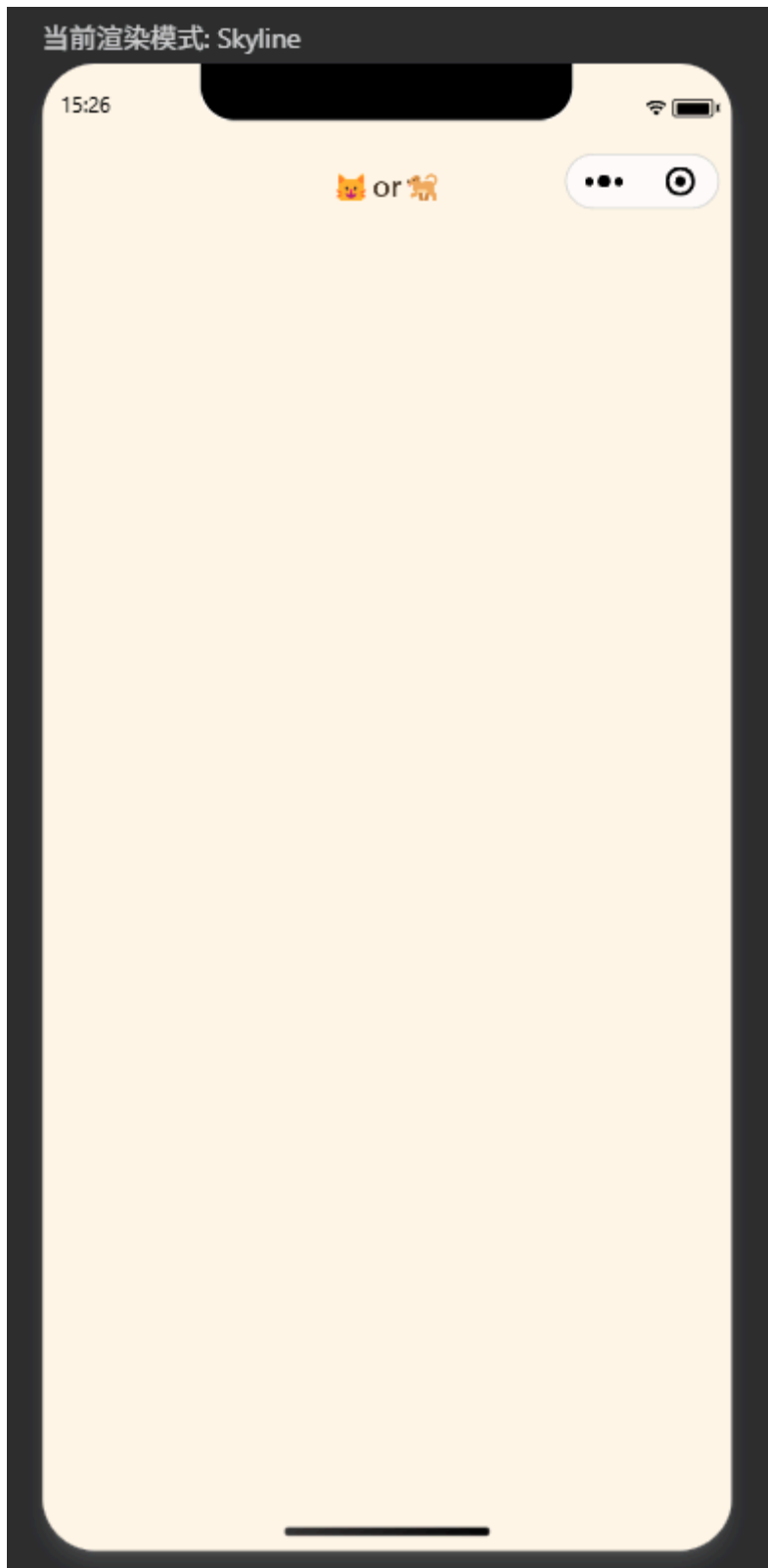
3.1 页面导航和整体结构

页面使用项目自带的 `navigation-bar` 组件设置顶部导航，并使用 `scroll-view` 作为可滚动的主体区域：

```
<navigation-bar
  title="🐱or🐶"
  back="{{false}}"
  color="#4a3828"
  background="#fff8ea">
</navigation-bar>

<scroll-view class="scrollarea" scroll-y enhanced
  show-scrollbar="{{false}}">
  <view class="page-container">
    <!-- 页面内容 -->
  </view>
</scroll-view>
```

`title` 设置导航栏标题，`back="{{false}}"` 表示当前页面是首页，不显示返回按钮。`scroll-y` 允许页面纵向滚动，避免内容在小屏幕设备中显示不完整；`show-scrollbar="{{false}}"` 隐藏滚动条，使页面更加简洁。



3.2 使用数据绑定显示当前宠物

在 `index.js` 的 `data` 中建立 `pets` 数组，每个元素同时保存问候文字、图片路径和对应表情。这样可以保证切换时文字和图片始终一一对应。

```
data: {
  pets: [
    { wording: 'hello dog', image: '/img/sausage-hotdog.jpg', emoji: '🐕' },
    { wording: 'hello orange cat', image: '/img/orange-cat.jpg', emoji: '🐱' },
    { wording: 'hello black cat', image: '/img/black-cat.jpg', emoji: '🐱•██' },
    { wording: 'hello white cat', image: '/img/white-cat.jpg', emoji: '🐱' }
  ],
}
```

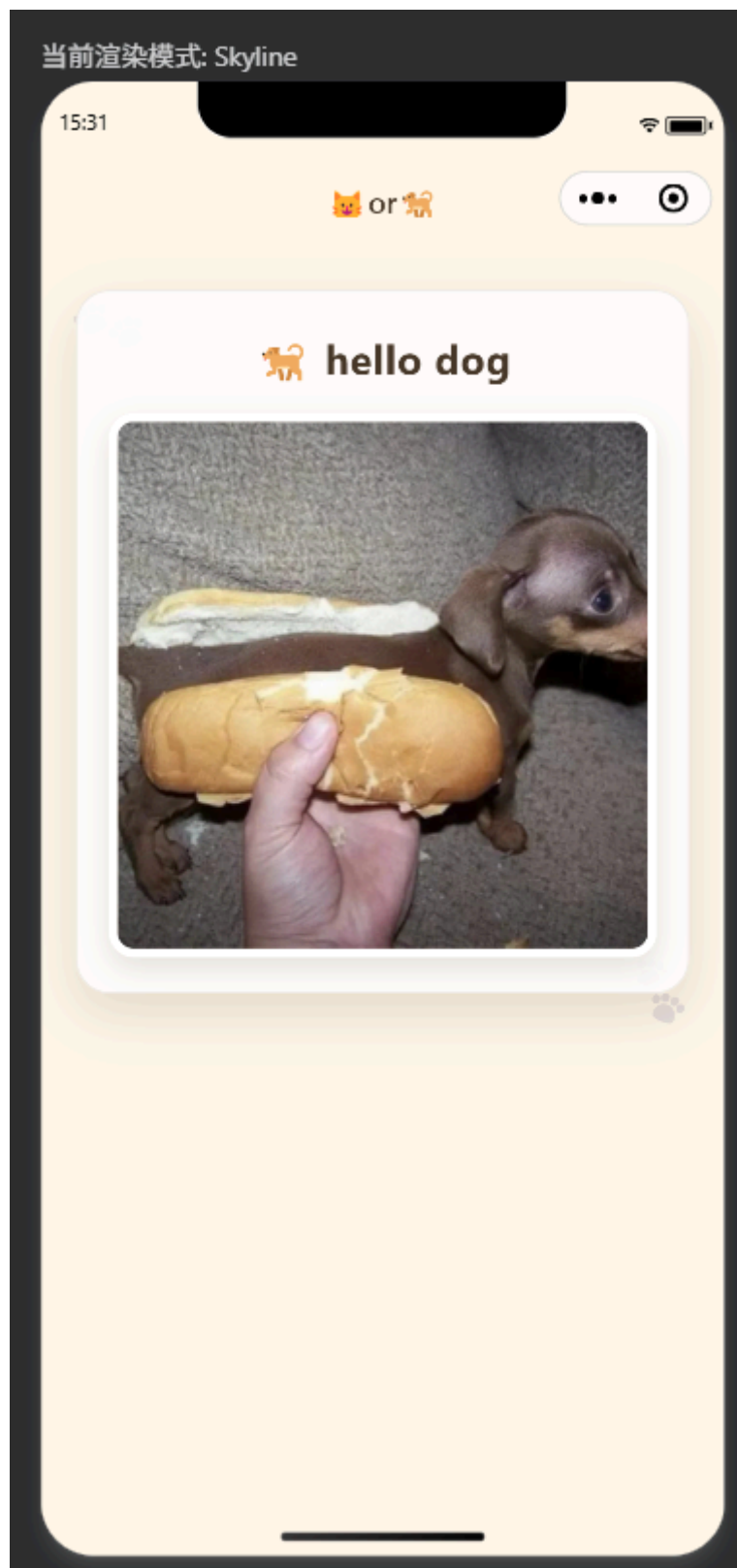
```
currentIndex: 0,  
currentPet: {  
  wording: 'hello dog',  
  image: '/img/sausage-hotdog.jpg',  
  emoji: '🐶'  
}  
}
```

`currentIndex` 保存当前宠物在数组中的下标，初始值为 0，所以程序第一次打开时显示数组中的腊肠犬。`currentPet` 保存当前需要展示的完整宠物对象。

WXML 使用双大括号进行数据绑定：

```
<view class="greeting-row">  
  <text class="pet-emoji">{{currentPet.emoji}}</text>  
  <text class="title">{{currentPet.wording}}</text>  
</view>  
  
<image class="pet-image"  
  mode="aspectFill"  
  src="{{currentPet.image}}">  
</image>
```

当 `currentPet` 发生变化时，页面中的表情、问候文字和图片会自动更新。`mode="aspectFill"` 让图片在保持原始比例的同时铺满固定大小的图片区域，多余部分自动裁剪



3.3 实现按钮点击和循环切换

按钮使用 `bindtap` 将点击事件绑定到 `changePet` 函数：

```
<button class="change-button"
  bindtap="changePet"
  hover-class="button-hover">
  not me
</button>
```

其中, `bindtap="changePet"` 表示点击按钮时执行 JavaScript 中的同名函数; `hover-class` 用于设置按钮按下时的视觉反馈。

循环切换逻辑如下:

```
changePet() {  
  const nextIndex =  
    (this.data.currentIndex + 1) % this.data.pets.length  
  
  this.setData({  
    currentIndex: nextIndex,  
    currentPet: this.data.pets[nextIndex]  
  })  
}
```

每次点击后, 当前下标加 1, 再对数组长度取模。

`this.setData()` 用来修改页面数据, 并通知小程序重新渲染相关 WXML 内容。不能只直接修改 `this.data`, 否则界面不会可靠地同步更新。

15:32



🐱 or 🐶



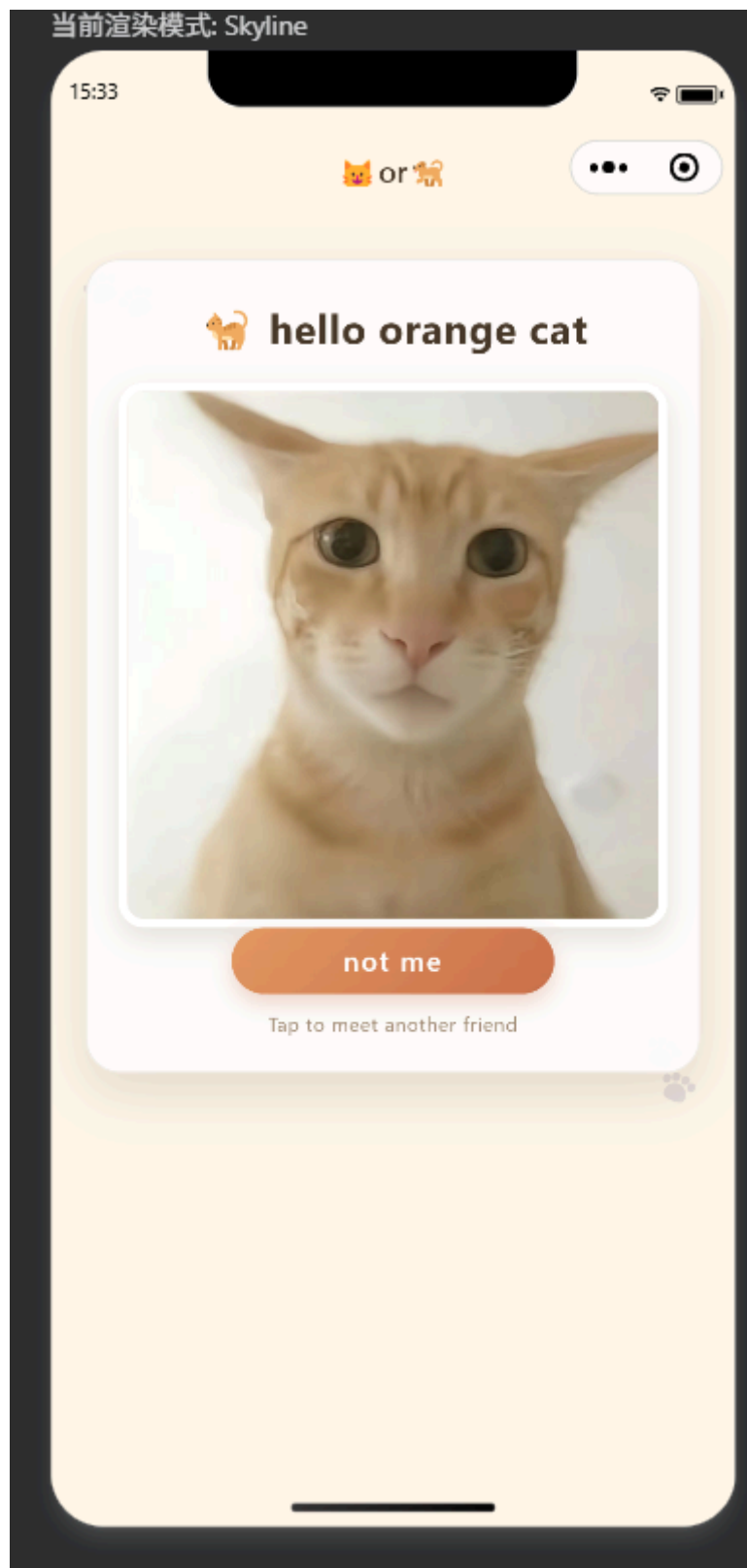
🐶 hello dog



not me

Tap to meet another friend





3.4 使用列表渲染制作状态指示点

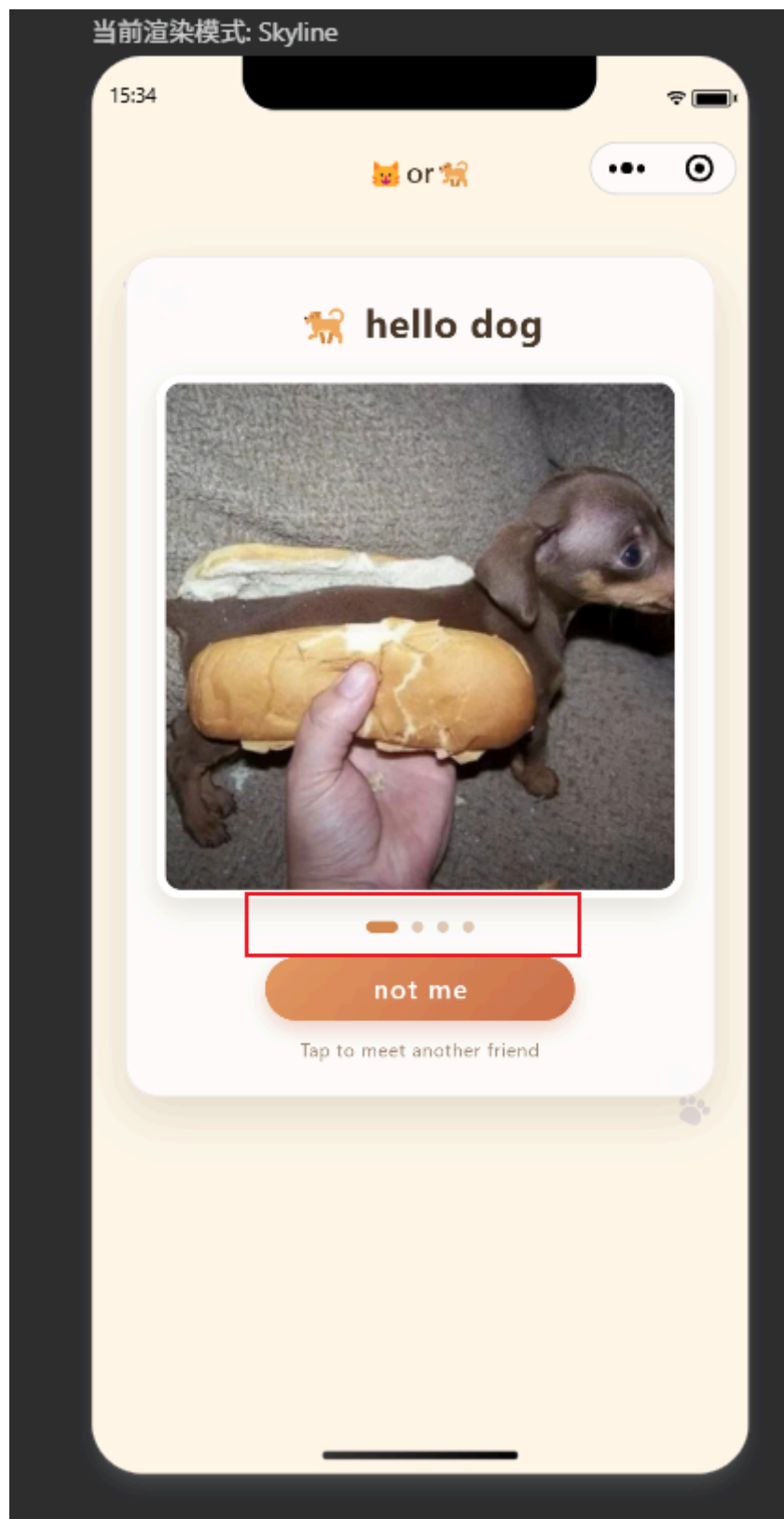
图片下方设置四个状态指示点，用来提示当前处于第几张图片：

```
<view class="indicator-list">
  <view
    wx:for="{{pets}}"
    wx:key="wording"
    class="indicator {{index === currentIndex
      ? 'indicator-active' : ''}}">
  </view>
</view>
```

`wx:for="{{pets}}"` 会根据宠物数组循环创建四个 `view`。系统自动提供的 `index` 表示当前循环下标；当它等于 `currentIndex` 时，元素会额外获得 `indicator-active` 样式，使当前指示点变长并改变颜色。

```
.indicator {
  width: 13rpx;
  height: 13rpx;
  margin: 0 8rpx;
  border-radius: 50%;
  background: #dccbb8;
}

.indicator-active {
  width: 36rpx;
  border-radius: 8rpx;
  background: #d48852;
}
```



3.5 背景和宠物卡片设计

页面整体使用米白、浅橙和棕色组成的暖色风格，与猫狗图片的轻松主题保持一致。背景由两个径向渐变和一个线性渐变叠加而成：

```
.scrollarea {  
  flex: 1;  
  overflow-y: auto;  
  background:  
    radial-gradient(circle at 12% 18%,  
      rgba(255, 197, 92, 0.24) 0,
```

```
    rgba(255, 197, 92, 0.24) 90rpx,  
    transparent 92rpx),  
    radial-gradient(circle at 88% 82%,  
    rgba(133, 196, 157, 0.20) 0,  
    rgba(133, 196, 157, 0.20) 130rpx,  
    transparent 132rpx),  
    linear-gradient(160deg, #fff8ea 0%, #f9edda 100%);  
}
```

主体内容放在半透明白色卡片中，并通过圆角、边框和阴影增强层次：

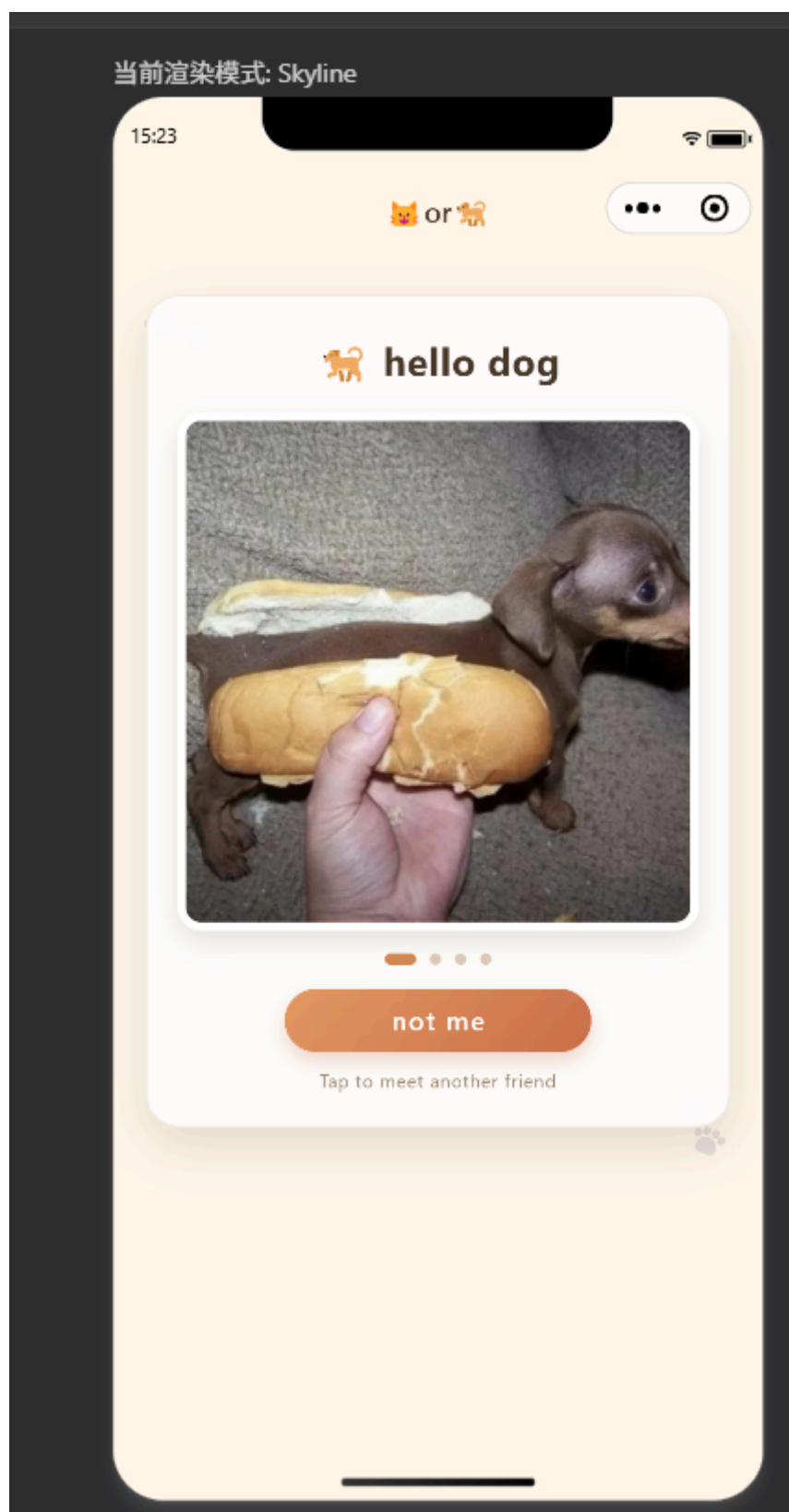
```
.pet-card {  
  width: 100%;  
  padding: 42rpx 34rpx 38rpx;  
  border: 2rpx solid rgba(151, 112, 75, 0.12);  
  border-radius: 38rpx;  
  background: rgba(255, 255, 255, 0.88);  
  box-shadow: 0 22rpx 55rpx rgba(106, 77, 48, 0.16);  
}
```

按钮采用胶囊形圆角和橙色渐变，并设置阴影与按压反馈：

```
.change-button {  
  width: 72%;  
  border: none;  
  border-radius: 999rpx;  
  background: linear-gradient(135deg, #e49a63, #cb704b);  
  color: #ffffff;  
  box-shadow: 0 10rpx 24rpx rgba(198, 105, 69, 0.28);  
}  
  
.button-hover {  
  opacity: 0.82;  
  transform: scale(0.98);  
}
```

页面尺寸主要使用微信小程序提供的 `rpx` 单位。屏幕宽度会被换算为 `750rpx`，所以图片、间距、文字和圆角可以根据不同设备宽度自动缩放。

4. 最终运行效果



15:23



🐱 or 🐱



🐱 **hello orange cat**



not me

Tap to meet another friend



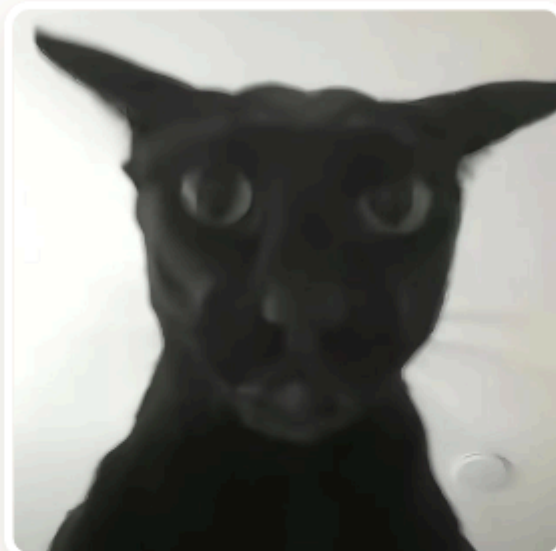
15:35



🐱 or 🐱



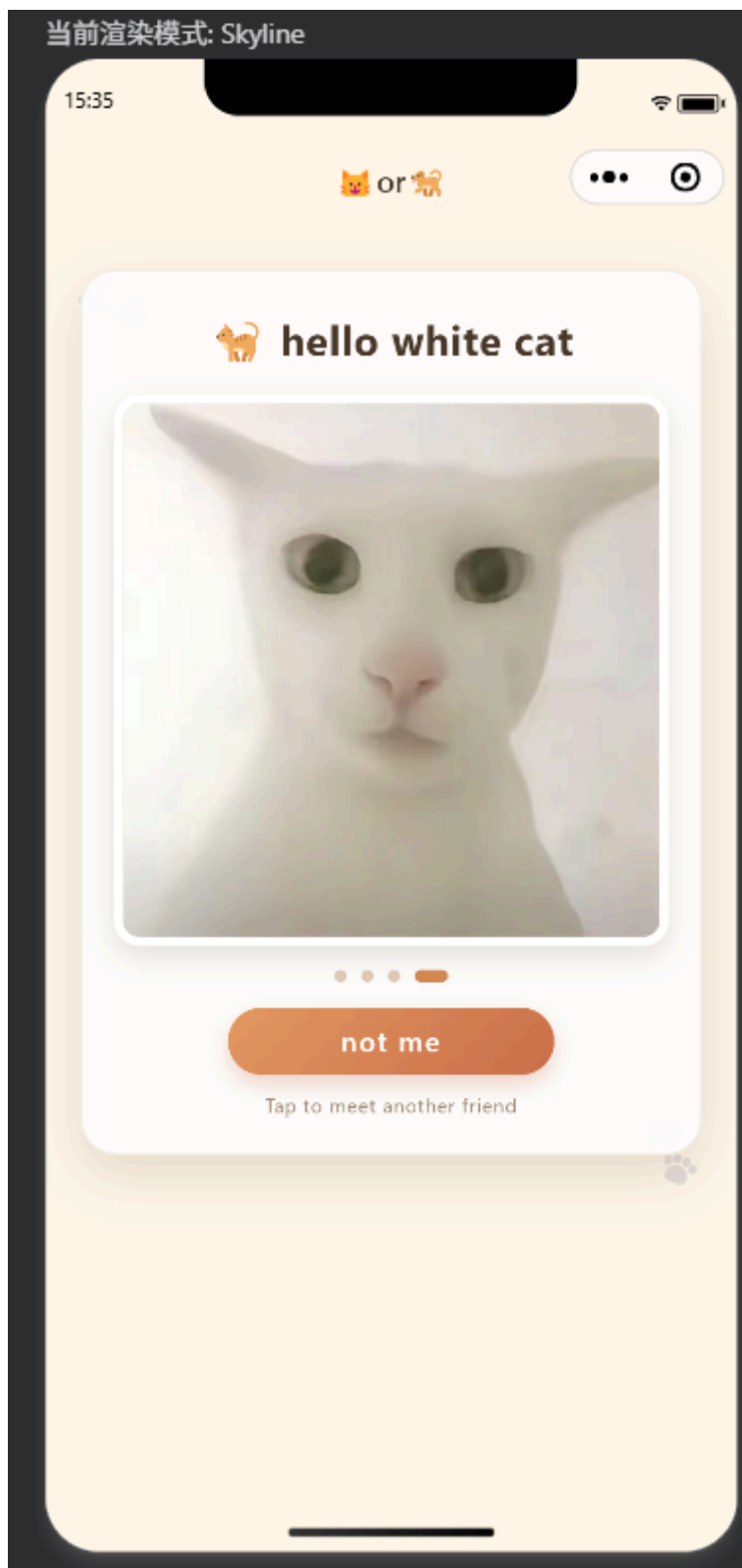
hello black cat



not me

Tap to meet another friend





二、问题总结与体会

1. 实验中遇到的问题及解决方法

(1) **不同图片尺寸导致页面布局变化。** 四张图片的宽高比例并不完全一致，如果使用 `widthFix`，切换图片时展示区域高度会发生改变。解决方法是在 `.image-frame` 中设置统一的宽高，再给图片设置 `width: 100%`、`height: 100%` 和 `mode="aspectFill"`。这样图片会保持比例并填满容器，切换时页面结构更加稳定。

(2) **两个状态的三元运算无法满足四个状态循环。** 基础案例可以通过三元表达式在 `girl` 和 `boy` 之间切换，但本实验需要依次展示四组文字和图片。使用对象数组统一保存每只宠物的数据，通过“下标加一后对数组长度取模”的方式完成循环。

(3) **顶部区域可能被状态栏遮挡。** 使用自定义 `navigation-bar` 组件处理系统状态栏和胶囊按钮区域，页面主体放在导航栏下方的滚动容器中，相比直接添加固定顶部内边距，更适合不同设备。

2. 实验收获与体会

通过本次实验，我熟悉了微信小程序的基本流程，理解了 WXML、WXSS、JavaScript 和 JSON 配置文件之间的分工。WXML 负责页面结构和数据绑定，WXSS 负责外观及移动端布局，JavaScript 负责数据与交互逻辑，JSON 负责页面配置和组件注册。